Skeptic Engine — Architecture Review and Interview Dossier

Reverse-engineered from source | v0.1.0 | Revised edition reflecting the current implementation

Prepared for Athitthiyan

Updated 6 July 2026 (original 30 June 2026)

Contents

Revision Notes — what changed since 30 June 2026

This edition re-verifies every claim against the current source (HEAD `cced496`; the original was reverse-engineered from `2c9cfca`). The codebase has advanced materially: a number of items the June review listed as "missing" or "scaffolded" are now implemented. Corrections are folded into every phase below; the headline deltas are:

Area	June 30 (old)	Now (verified in code)	Evidence
Observability / metrics	"No `/metrics` endpoint; observability weak"	`/metrics` exposed via prometheus-fastapi-instrumentator; optional LangSmith tracing; per-request and per-LLM-call telemetry tables	`app/main.py`, `config.py` (`METRICS_ENABLED`, `LANGSMITH_TRACING`)
CI/CD	"No pipeline in repo"	GitHub Actions pipeline: ruff + pytest, eslint + tsc + vitest + build, GHCR image publish, SSH deploy	`.github/workflows/ci-cd.yml`

Area	June 30 (old)	Now (verified in code)	Evidence
LLM resilience	"Single vendor; no fallback, cache, retry, or circuit breaker"	Multi-provider gateway (Anthropic, Groq, OpenAI, Gemini, DeepSeek) with fallback order, response cache, tenacity retry on transient errors, cost/token telemetry	`app/llm/` (`service.py`, `cache.py`, `routing.py`, `pricing.py`, providers)
Rate limiting	"None (S5, High)"	Rate-limit module applied on auth	`rate_limit.py`, `routes/auth.py`
Authorization (RBAC)	"Role never enforced anywhere (S1, Critical)"	`require_elevated_role` (partner/admin) enforced on destructive investigation routes; still not on review approve/reject/escalate	`security.py`, `investigations.py`
RAGAS evaluation	"Proxy-only from telemetry"	Real-time LLM-judge scoring (ChatAnthropic judge), results persisted	`RAGAS_REALTIME_ENABLED`, `app/evaluation/`, `ragas_evaluation_results` table
RAG / retrieval	"Scaffolded, not built; no chunking/embeddings/search"	Working hybrid retriever: chunking + deterministic feature-hash embeddings + cosine and lexical ranking, indexed at startup, `/knowledge/search` and reindex; still deterministic (not a learned model) and stored as JSON, not a pgvector ANN index	`app/knowledge/retriever.py`
Global error handler	"Should add one + correlation IDs"	Global handler returns a generic message plus `request_id`; internals hidden unless `DEBUG`	`app/main.py`
Schema authority	"`create_all` and Alembic both run (split authority)"	Production skips `create_all`; Alembic-only in prod	`app/main.py` lifespan
Deploy artifacts	"Docker + k8s only; manual"	Added production Compose stack and Railway config-as-code (api/worker/beat), plus the k8s manifest	`compose.production.yml`, `railway.*.json`
Data model	"10 tables"	14 tables; 5 Alembic revisions	`models.py`, `migrations/`
Routers	"12 routers"	14 routers (adds `intake`, `agents`, `settings`)	`app/api/routes/`
Real agents	"Flag-gated off; graph compiled but unused"	Real LangGraph nodes invoked when `USE_REAL_AGENTS=true` (off-thread); still default-off	`app/agents/executor.py`

Production-readiness score: 69/100 (was 58/100). Still pilot-grade, but the security, observability, AI-resilience, and delivery gaps that dominated the June review are substantially narrower. Remaining hard gaps: comprehensive RBAC, browser-exposed credentials, prompt-injection defense, learned embeddings + pgvector ANN, distributed tracing, and automated backup/DR. Detail in Phase 13 and Phase 16.

Executive Summary

Skeptic Engine (internal package name `professional-skepticism-engine`) is a real-time, multi-agent AI audit-investigation platform. It ingests a financial-ledger transaction and runs it through a crew of six cooperating and adversarial LLM agents — Supervisor, Evidence, Challenger, Defender, Adjudicator, Verifier — that debate the transaction's risk from opposing sides, ground every claim in retrieved evidence, route uncertain cases to humans, and write every step to a tamper-evident audit trail, streaming progress live to the UI over WebSockets.

The name encodes the thesis: professional skepticism — the auditing principle that conclusions must be evidenced and challenged, not assumed — implemented as software. The architecture's defining choices all serve trust over cleverness: adversarial debate instead of a single prompt, a separate Verifier gate instead of blind trust, hash-chained immutable logs instead of plain rows, no-fabrication external benchmarks, and human escalation when the machine is not confident.

Verdict at a glance. The codebase is unusually well-structured for a v0.1.0: a clean modular monolith with crisp domain seams, dual-path graceful degradation (works on a laptop and scales on Kubernetes via feature flags), real tests, Alembic migrations, multi-stage Docker, a full k8s manifest, a production Compose stack, and a CI/CD pipeline. It is demo/pilot-ready and now approaching — but not yet at — enterprise-production readiness. The principal remaining gaps are: RAG uses deterministic feature-hash embeddings rather than a learned model and a real vector index; the real-agent path is still flag-gated off by default (`USE_REAL_AGENTS=false`); authorization (RBAC) is enforced only on destructive investigation routes, not review actions; the Next.js app still exposes dev credentials via `NEXT_PUBLIC_*`; there is no prompt-injection defense; and observability has metrics but no distributed tracing. Production-readiness score: **69/100** (detailed scorecard in Phase 13).

Dimension	State	One-line
Architecture	Strong	Modular monolith, LangGraph agent core, clean layering
AI design	Strong intent, mostly implemented	Multi-agent debate + grounding verifier; multi-provider LLM with fallback/cache/retry; real-time RAGAS judge; hybrid retrieval (deterministic embeddings); real agents flag-gated off by default
Backend	Strong	FastAPI, SQLAlchemy 2.0, Celery, dual-path execution, global error handler
Frontend	Strong	Next.js 15 / React 19, React Query, typed service layer
Data	Good	14 tables, indexed, hash-chained audit; embeddings as JSON; no soft-delete
Security	Fair (was weak)	Rate limiting + partial RBAC now present; client-exposed creds, no prompt-injection defense, plain-env secrets remain
Observability	Fair (was weak)	`/metrics` + optional LangSmith + request/LLM telemetry; no OTel distributed traces
Delivery	Good (was none)	CI/CD pipeline, production Compose, Railway config, k8s manifest

Dimension	State	One-line
Production readiness	Partial	69/100 — pilot-ready, hardening in progress

Phase 1 — Repository Discovery

1.1 Top-level layout

The repository is a polyrepo-in-a-monorepo: two independently runnable applications plus prototype, docs, and delivery tooling.

```
Skeptic Engine/
  Backend/     FastAPI + Python 3.11 --- the "brain" (agents, pipeline, data, audit)
  UI/          Next.js 15 / React 19 --- the "cockpit" (live updates)
  Prototype/   Original single-file HTML PoC (PSE_Prototype.html)
  Docs/        PRD, HLD/LLD, design PDFs, sample outputs, and the enterprise docs set
  .github/workflows/ci-cd.yml   CI/CD pipeline (NEW since June)
  docker-compose.production.yml  Production stack (NEW)
  DEPLOYMENT_GUIDE.md, PROJECT_ANALYSIS.md, LICENSE (MIT)
```

The two apps communicate over HTTP and WebSocket only — no shared code, no shared database driver, no in-process coupling. That seam is the single most important structural decision: either half can be built, tested, deployed, or replaced alone.

1.2 Backend module map (`Backend/app/`)

```
app/
  main.py                 create_app() factory + lifespan; CORS + TrustedHost;
                          /metrics instrumentation; global exception handler;
                          Alembic-only schema in production
  core/
    config.py             pydantic-settings Settings (env-aware, prod safety checks)
    security.py           bcrypt + JWT (HS256) + get_current_user + require_elevated_role
    request_logging.py    per-request audit middleware -> request_logs
    rate_limit.py         auth rate limiting (NEW)
  api/routes/             14 routers: health, auth, intake, investigations, claims,
                          reviews, agents, analytics, evaluation, knowledge, reports,
                          audit, settings, websocket
  schemas/                Pydantic request/response DTOs
  db/
    models.py             14 SQLAlchemy ORM models + enums
    session.py            engine, SessionLocal, get_db_session, pool tuning
  agents/
    crew.py               LangGraph 6-agent state machine + node factories
    executor.py           InvestigationExecutor --- pipeline + control loop + streaming
  llm/                    NEW: multi-provider gateway
    service.py            routing, fallback, cache, tenacity retry, telemetry
    routing.py, cache.py, pricing.py, tokenization.py, settings_store.py
    providers/            anthropic, groq, openai, gemini, deepseek
  knowledge/retriever.py  hybrid retrieval: chunking + feature-hash embeddings + ranking
  realtime/
    websocket_manager.py  in-process connection pool + typed event classes
    redis_bus.py          cross-process pub/sub bridge (worker -> API)
  tasks/celery_app.py     Celery app, tasks, backoff retries, beat schedule
  evidence_verification/  third-party benchmark verification (no fabrication)
  evaluation/             RAGAS real-time LLM-judge scoring (NEW: real judge)
  audit/eventstore.py     EventStoreDB primary + Postgres hash-chain fallback
```

1.3 Frontend module map (`UI/`)

Unchanged in shape from the June review: `app/(app)/` route group over a shared `AppShell`; `features/` screen compositions; `components/` domain folders + a shadcn-style `ui/` primitive set; `hooks/` React Query wrappers including `use-investigation-realtime`; `services/` typed clients with an `api.ts` fetch core and snake_case-to-camelCase mappers; `store/ui-state.tsx` React Context for UI-only state; `types/domain.ts`; `constants/` route table and navigation; `lib/` utilities. An `unauthorized` route segment now exists.

1.4 Runtime, build, and tooling

Concern	Backend	Frontend
Language / runtime	Python 3.11	TypeScript 5.8 / React 19
Framework	FastAPI 0.115 + uvicorn 0.34	Next.js 15.3 (App Router)
Package manager	pip + pyproject.toml	pnpm 9.15
ORM / data	SQLAlchemy 2.0.36, Alembic 1.14, psycopg2	—
Agent / LLM	LangGraph 0.2.60, langchain-anthropic 0.3.1, anthropic 0.42; multi-provider gateway	—
Async / queue	Celery 5.4 + Redis 5.2, Flower	—
Eval	RAGAS 0.2.15 (real-time LLM judge)	—
Observability	prometheus-fastapi-instrumentator 7.0, prometheus-client, langsmith 0.2	—
Validation	Pydantic 2.10 / pydantic-settings 2.7	Zod 3.25 + react-hook-form 7.58
Server state	—	TanStack React Query 5.80
Tables / charts / graph	—	TanStack Table 8, Recharts 2.15, ReactFlow 11
Tests	pytest 8.3 (+asyncio, +cov), httpx	Vitest 3.2, Testing Library, jsdom
Lint / format	ruff 0.8.6 (CI-enforced)	ESLint 9, Prettier 3.5, Husky + lint-staged
Container / orchestration	Multi-stage Dockerfile, dev + production Compose, k8s manifest, Railway config	Multi-stage Dockerfile, standalone Next build
CI/CD	GitHub Actions (NEW)	same pipeline

Infra now present (correcting the June "missing" list): a CI/CD pipeline (`.github/workflows/ci-cd.yml`) and a `/metrics` endpoint both exist. **Still missing:** Terraform/Pulumi IaC (the k8s YAML, production Compose, and Railway JSON are the deploy artifacts) and OpenTelemetry distributed tracing.

1.5 Configuration and secrets

All backend config flows through one `pydantic-settings` class (`app/core/config.py`) reading `.env`. Feature flags gate every heavy dependency: `USE_REAL_AGENTS`, `USE_CELERY`, `USE_REDIS_EVENTS`, `USE_EVENTSTORE` default off; `AUTH_REQUIRED` defaults off in dev but is **forced on when `ENV=production`**; `SEED_DEFAULT_USER` defaults on. New flags include `METRICS_ENABLED`, `LANGSMITH_TRACING`, `ENABLE_LLM_FALLBACK`, `LLM_CACHE_ENABLED`, and `RAGAS_REALTIME_ENABLED`. Frontend config is `NEXT_PUBLIC_*` (API base URL

and — still a problem — username/password for the dev password-grant flow, which ships to the browser bundle). Secrets are plain `.env` files (with `.example` templates) plus a production `.env.production` on the host and GitHub Actions Secrets for CI; there is still no secrets manager (Vault/SSM/Sealed-Secrets); k8s uses a raw Secret.

Phase 2 — Business Understanding

2.1 The problem

External and internal audit teams must review huge volumes of financial transactions for fraud, policy violations, and control failures. Manual review is slow, inconsistent, and expensive; naive "ask an LLM if this is fraud" automation is dangerous because a single model hallucinates, anchors on one narrative, and produces conclusions an auditor cannot defend to a regulator. Audit work has a non-negotiable requirement: every conclusion must be evidenced, challenged, and traceable. Skeptic Engine automates the investigation while preserving that evidentiary discipline.

2.2 Users and stakeholders

- **Analyst** — submits/imports transactions (case intake), watches the live investigation, triages.
- **Reviewer** — works the human-review queue: approve, reject, request more evidence.
- **Engagement Partner** — final sign-off authority; receives escalations and critical-risk cases.
- **Audit-firm / enterprise compliance (buyer)** — pays for throughput, consistency, and an immutable audit trail that survives regulatory scrutiny (SOX/PCAOB-style).

The role vocabulary is in code: `UserCreate.role` is `Literal["analyst","reviewer", "partner","admin"]`; `ELEVATED_ROLES = {"partner","admin"}` now gates destructive actions.

2.3 Why someone pays

(1) **Scale** — agents triage thousands of cases so humans focus on the genuinely ambiguous; (2) **defensibility** — the hash-chained audit log plus step-by-step replay means every verdict can be reconstructed and proven un-tampered; (3) **consistency** — the same skeptical pipeline applied uniformly; (4) **grounding guarantees** — the Verifier blocks ungrounded verdicts, and the third-party service refuses to fabricate benchmarks.

2.4 End-to-end user journey (real scenario)

An analyst imports a ledger. A USD 92,400 payment to "Apex Logistics" coded to "Consulting" is above the USD 50,000 materiality threshold.

1. **Intake** — analyst submits the case (`/intake`). Backend persists it, writes a `case_created` audit event, and auto-runs third-party evidence verification.
2. **Execute** — analyst hits Run; the case enters the agent pipeline (Celery or in-process).
3. **Evidence** — Evidence agent gathers ledger metadata, materiality context, red flags, and (now) retrieved policy-knowledge context.
4. **Debate** — Challenger argues an unapproved control risk; Defender argues a traceable, in-policy spend. Up to two rounds, live-streamed to the Debate Viewer.
5. **Adjudication** — Adjudicator returns `{risk: high, confidence: 0.78, key_concerns:[...]}`.
6. **Verification** — Verifier finds the high-risk verdict rests on a materiality overage with no external corroboration, and marks it ungrounded.
7. **Supervisor re-run** — the case loops back; Evidence re-queries, finds an approved supplier and PO. Re-adjudicated to medium at 0.92, now grounded.
8. **Confidence gate** — medium risk still trips review rules, so it routes to the reviewer queue.
9. **Human review** — reviewer reads the transcript, evidence, and verdict, approves; status becomes closed, audit event written.
10. Every phase is a checkpoint enabling Replay to reconstruct the case frame-by-frame.

Login is `POST /auth/token` (OAuth2 password grant) yielding a Bearer JWT attached to every call by `services/api.ts`.

Phase 3 — Architecture Walkthrough

3.1 What architecture this is

Skeptic Engine is a **modular monolith** with a layered and lightly hexagonal internal structure, wrapping an **agentic-AI workflow engine** (LangGraph state machine), with **event-streaming** for realtime and **event-sourcing-flavored audit** plus checkpointing:

- **Modular monolith** — one API process and one worker process, internally split into high-cohesion packages (`agents`, `llm`, `knowledge`, `realtime`, `audit`, `evidence_verification`, `evaluation`, `db`, `api`) with explicit import seams.
- **Layered** — `api/routes` (controllers) to `agents/executor` (domain) to `db` (persistence) to external adapters (`eventstore`, `redis_bus`, LLM providers, evidence providers).
- **Hexagonal touches** — heavy adapters (LLM, EventStoreDB, Redis, providers) are lazy-imported behind functions/flags so the core boots and tests run with none present.
- **Agentic workflow** — the business logic is a graph of agents with a supervisor control loop (Phase 8).
- **Event-driven realtime** — executor emits typed events to Redis pub/sub plus in-process broadcast.
- **Event-sourcing-flavored audit** — append-only hash-chained `audit_log` plus per-phase state checkpoints enabling deterministic replay.

3.2 Why this architecture (trade-off ledger, updated)

Why modular monolith (not microservices), why not pure CQRS, and why not serverless: the June reasoning stands — one team, correlated change cadence, a shared domain model, and a horizontally-scalable Celery worker tier for the one component (agents) that needs independent scale. The updated trade-off ledger:

Axis	This design	Cost / caveat
Cost	Low infra footprint; one image, two process types	Worker tier idle cost when no cases
Complexity	Moderate; clear layers	Dual-path (flagged) code adds branches to test
Scalability	API scales to 3+ replicas; workers scale horizontally	Single Postgres is the eventual bottleneck
Maintainability	High cohesion, lazy adapters, typed DTOs both ends	Stub/real divergence can drift
Latency	Realtime via WS; agents async off the request path	LLM latency dominates
Dev productivity	Runs fully offline (flags off), real tests	Two languages, two toolchains
Testing	pytest with sqlite + stub agents	Stubs do not exercise real LLM behavior
Deployment	Docker + k8s + production Compose + Railway + CI/CD	No Terraform/IaC yet
Observability	Metrics endpoint + request/LLM telemetry + optional LangSmith	No OTel distributed traces yet

Phase 4 — Folder-by-Folder Walkthrough

`app/core/` — cross-cutting concerns: `config`, `security` (now including `require_elevated_role`), `request_logging` (per-request audit), and `rate_limit` (auth throttling). Improvement still open: a `deps.py` for DI and broader RBAC coverage.

`app/api/routes/` — 14 routers (adds `intake`, `agents`, `settings` since June), one per domain, under `/api/v1`. Thin orchestration; some routes still embed SQLAlchemy queries directly (no repository layer — Phase 16, R10).

`app/schemas/` — Pydantic DTOs decoupling the wire contract from ORM rows (`from_attributes`, `use_enum_values` so enums serialize as strings).

`app/db/` — `models.py` (14 tables + enums) and `session.py` (engine, pooling, dialect-aware hooks; SQLite vs Postgres branch so tests run on sqlite and prod on pooled Postgres).

`app/agents/` — the domain core. `crew.py` defines the LangGraph nodes and graph; `executor.py` runs the pipeline, owns the Supervisor retry loop, persistence, checkpointing, and event emission. Real nodes are invoked when `USE_REAL_AGENTS=true`.

`app/llm/` (new) — the multi-provider gateway: `service.complete()` routes to the default provider, falls back through `LLM_FALLBACK_ORDER`, checks a response cache, retries transient provider errors with tenacity exponential backoff, and records cost/latency/tokens to `llm_call_logs`. Providers for Anthropic, Groq, OpenAI, Gemini, DeepSeek. Pricing and tokenization are modularized.

`app/knowledge/retriever.py` (now implemented) — a working hybrid retriever: it chunks the policy corpus (`_chunk_text`), builds stable feature-hash embeddings (`embed_text`, SHA-256 token hashing), ranks by clamped cosine plus token-overlap lexical score (`_rank_chunks`), and upserts chunks into `vector_embeddings` (`sync_knowledge_embeddings`, run at startup). `retrieve_knowledge_context` feeds the Evidence agent. Caveat: embeddings are deterministic (not a learned model) and stored as JSON, ranked in Python — not a pgvector ANN index (Phase 16, R11).

`app/realtime/` — `websocket_manager.py` (per-investigation connection pool + typed events) and `redis_bus.py` (cross-process bridge). The worker cannot reach the API's in-memory WS connections, so it publishes to Redis and the API forwards.

`app/tasks/` — Celery app, business tasks (`execute_investigation`, `collect_evidence`, `generate_report`, `check_materiality`, `score_investigation_ragas`) with max-retries and exponential backoff, plus a beat schedule (`cleanup_old_states`, `sync_vector_embeddings`). Evidence collection is now backed by the knowledge retriever; report generation remains basic.

`app/evidence_verification/` — a properly-layered service: normalizes a claim, picks a configured HTTP provider, applies per-category tolerance (FX, flight, hotel, food, cab, fuel, GST), and persists a `ThirdPartyEvidenceVerification`. Design rule: it never fabricates benchmark amounts; missing provider config yields `API_UNAVAILABLE`.

`app/evaluation/` (now real) — real-time RAGAS scoring: an Anthropic judge scores each investigation and results persist to `ragas_evaluation_results` and surface at `/evaluation`.

`app/audit/` — `eventstore.py`: EventStoreDB primary (lazy `esdbclient`), Postgres SHA-256 hash-chain fallback.

Frontend — pages are thin and delegate to `features/`; `features/` compose `components/` and `hooks/`; `hooks/` are the React Query service-orchestration layer; `services/` are the anti-corruption boundary through `api.ts`; server state lives in React Query, UI-only state in a small Context.

Phase 5 — Key File Walkthrough

`Backend/app/main.py` — `create_app()` factory plus lifespan. On startup it seeds a default user, indexes the knowledge base, and best-effort connects EventStoreDB (warns and falls back on failure); **in production it skips `create_all` and relies on Alembic**; on shutdown it disposes the engine. It adds CORS and Trusted-Host middleware, request logging, **Prometheus instrumentation at `/metrics`** (when `METRICS_ENABLED`), includes all 14 routers, and registers a **global exception handler** that returns a generic message plus a `request_id` (internals only shown when `DEBUG`).

`app/core/config.py` — `Settings` plus a `@model_validator _production_safety_checks`: refuses to boot in `ENV=production` with a default/short secret key, wildcard CORS/hosts, or a missing admin password; requires the provider API key when `USE_REAL_AGENTS=true`; and requires a LangSmith key when tracing is on. Fail-fast config validation — a standout.

`app/core/security.py` — bcrypt with a SHA-256 pre-hash (sidestepping bcrypt's 72-byte truncation), JWT issue/verify, `get_current_user`, and `require_elevated_role` (partner/admin). Caveat: when `AUTH_REQUIRED=false` (dev), `get_current_user` returns anonymous and role is not enforced; production forces auth on.

`app/llm/service.py` (new) — the LLM orchestration core: provider routing, fallback, response cache, tenacity retry on transient errors, and cost/token telemetry. Single choke point for all model calls — the resilience layer the June review said was missing.

`app/agents/executor.py` — `InvestigationExecutor`, the heart of the runtime. `execute_investigation()` runs the bounded Supervisor loop (evidence, debate, adjudication, verification), re-running on ungrounded verdicts up to `MAX_VERIFICATION_RETRIES`, then escalates; on grounded, a confidence gate routes to report or human review. Each phase writes DB rows, emits events both ways, and checkpoints state with a SHA-256 hash. A full deterministic stub path mirrors the real pipeline for offline demos.

`app/knowledge/retriever.py` (new) — hybrid retrieval described in Phase 4.

`app/api/routes/investigations.py` — CRUD plus `/execute`, `/replay`, `/workspace`, `/stats/summary`. `/execute` embodies dual-path degradation: try Celery, preflight the broker (0.25s ping), else fall back to a FastAPI background task running the executor inline. Destructive routes now depend on `require_elevated_role`.

`UI/services/api.ts` — the fetch core: resolves a token, attaches Authorization, parses by content-type, throws typed errors, and derives the ws/wss URL. Single choke point for all backend I/O.

`UI/hooks/use-investigation-realtime.ts` — opens the WS and, on each event, invalidates the relevant React Query keys so the UI refetches (push-to-invalidate), with reconnect-backoff.

Phase 6 — End-to-End Execution Flow

1. User submits a case (intake form) via `features/intake` to `services/intake.service`.
2. `services/api.ts` attaches the Bearer JWT (password-grant or static token).
3. `POST /api/v1/intake` persists the case, writes a `case_created` audit event, and triggers third-party evidence verification.
4. `POST /api/v1/investigations/{id}/execute` enters the pipeline: try Celery (broker preflighted), else run inline as a background task.
5. The executor runs the Supervisor loop; each phase calls the LLM gateway (real) or a stub, writes DB rows, checkpoints state, and emits events.
6. Events flow worker to Redis pub/sub to the API WebSocket endpoint to the browser, where they invalidate React Query keys and the UI refetches.
7. On completion the confidence gate routes to report or the review queue; the case closes with a final audit event. Replay reconstructs frames from state checkpoints.

Phase 7 — Technology Decision Record (updated)

Area	Choice	Why it was right	Alternative and why not / status
Agent framework	LangGraph	Graph-native state and conditional edges; a debate loop plus verifier-driven re-run needs explicit control	CrewAI/AutoGen hide the control; Semantic Kernel is .NET-leaning
LLM	Anthropic Claude default, **multi-provider gateway**	Strong reasoning and structured-output adherence; tiered Sonnet/Haiku	OpenAI/Gemini/Groq/DeepSeek now first-class with fallback order, cache, and retry (the June "no fallback yet" gap is closed)
Relational DB	PostgreSQL 16	ACID for audit integrity, JSON columns, pgvector available	Mongo weaker on multi-row integrity; MySQL lacks vector/JSONB parity
Broker / cache / bus	Redis 7	One dependency for Celery broker + result + pub/sub + cache	RabbitMQ/Kafka are ops weight not needed yet
Task queue	Celery	Mature retries/backoff/beat, Flower monitoring	Temporal heavier; Arq/Dramatiq smaller ecosystem
Audit store	EventStoreDB + PG hash-chain	Purpose-built append-only streams; fallback gives tamper-evidence without it	Kafka is not a queryable audit store; a plain table is not tamper-evident
AuthN	JWT (HS256) via OAuth2 password	Stateless, fits SPA + WS	OIDC/IdP is the enterprise endgame, not yet wired
Authorization	`require_elevated_role` (partial)	Gates destructive investigation routes now	Not yet on review actions — comprehensive RBAC pending (R1)
Retrieval	Hybrid feature-hash + lexical	Working retrieval without an embedding service dependency	Learned embeddings + pgvector ANN (IVFFlat/HNSW) still pending (R11)
Observability	prometheus-fastapi-instrumentator + LangSmith	Metrics and optional tracing with minimal surface	OpenTelemetry distributed traces still pending (R4)
CI/CD	GitHub Actions	Lint/test/build/publish/deploy in one pipeline	Argo/Helm canary is the scale-up path
API style	REST + WebSocket	CRUD is REST-shaped; realtime needs server-push	GraphQL/gRPC add friction without payoff

Area	Choice	Why it was right	Alternative and why not / status
Container	Docker (multi-stage) + k8s + Compose + Railway	Small non-root image, healthcheck; multiple deploy targets	Bare VMs lose orchestration

Calibration note: several "choices" (Claude model, thresholds, debate rounds, fallback order) are configuration, not hard-coding. Retrieval now exists but uses deterministic embeddings — in an interview, say "hybrid feature-hash retrieval implemented; learned-embedding + ANN upgrade is the next step," not "full semantic RAG."

Phase 8 — AI / GenAI Deep Dive

8.1 The multi-agent design (what is actually implemented)

Six agents wired as a LangGraph `StateGraph` over a shared `InvestigationState` TypedDict:

`supervisor -> evidence -> challenger <-> defender -> adjudicator -> verifier -> END`

- **Supervisor** — router/orchestrator: a deterministic state machine plus, in `executor.py`, the outer control loop (re-run on ungrounded).
- **Evidence** — retrieval/summarization; on retries it issues a feedback-aware corroboration query, and now pulls context from the knowledge retriever.
- **Challenger / Defender** — adversarial debate: same evidence, opposite priors, up to N rounds, each told to advance rather than repeat.
- **Adjudicator** — structured output: returns a JSON verdict `{risk_level, confidence, reasoning, key_concerns, mitigating_factors}`, parsed by `_extract_json`.
- **Verifier** — reflection/evaluator (grounding guard): checks material claims against evidence, returns `{is_grounded, ungrounded_claims, verification_report}`; runs on the cheaper Haiku tier at low temperature.

8.2 Prompt engineering present

Zero-shot role-conditioned prompts; chain-of-thought-ish instructions; structured JSON output for Adjudicator and Verifier with a defensive parser (`_extract_json`: `json.loads`, then a regex object match, then a safe default); iterative refinement carrying verification feedback into the next prompt; and model tiering (reasoning vs lightweight) for cost/latency. Provider and model are configurable per environment.

8.3 Hallucination prevention and grounding (the core IP)

Three independent guards: (1) adversarial debate surfaces both narratives before a verdict; (2) the Verifier can reject an ungrounded verdict and force a bounded re-run with a corroboration query before any human sees it; (3) the third-party evidence service validates claimed amounts against real external benchmarks and refuses to invent them. Grounding by construction, not a single "be accurate" instruction.

8.4 What is now implemented vs still pending (be precise in interviews)

Now implemented (was listed as missing in June):

- **Retrieval** — chunking, deterministic feature-hash embeddings, and hybrid cosine-plus-lexical ranking over a policy knowledge base, indexed at startup and queryable at `/knowledge/search`. It is real retrieval, not a placeholder.
- **Multi-provider resilience** — fallback order across five providers, a response cache, and tenacity retry on transient errors, with per-call cost/token telemetry.
- **Evaluation** — real-time RAGAS scoring via an Anthropic LLM judge, persisted per case.

Still pending (be honest):

- Embeddings are deterministic feature-hashing, not a learned embedding model, and are stored as JSON, not a pgvector ANN index — so retrieval quality and scale are limited (R11).
- No tool/function calling — agents return text/JSON, not LLM tool calls.
- Real agents are flag-gated off by default (`USE_REAL_AGENTS=false`); the default runtime uses deterministic stubs.
- No prompt-injection defense — vendor/category and external text still flow into prompts unsanitized (Phase 12, S6).

8.5 Human-in-the-loop and guardrails

The confidence gate is the guardrail: risk in critical/high/medium, confidence below `DEFAULT_CONFIDENCE_THRESHOLD` (0.7), or a flagged/unavailable third-party status routes to a human queue, with partner-vs-reviewer assignment by severity. Escalation triggers when the retry budget is exhausted. Appropriately conservative for audit (routes medium to review too — high recall, a tunable business decision).

Phase 9 — Database Walkthrough

9.1 Model and relationships

`investigations` is the aggregate root. Cascade children: `investigation_states` (checkpoints), `debate_transcripts`, `evidence_artifacts`, `verification_claims`, `third_party_evidence_verifications`, `review_queue`. `audit_log` references investigations logically (no FK) — correct for an append-only ledger that must survive parent deletion. Standalone/support tables: `users`, `vector_embeddings`, `request_logs` (HTTP audit), `runtime_settings` (editable governance singleton), `llm_call_logs` (cost/latency/token telemetry), `ragas_evaluation_results` (quality scores). That is **14 tables** (was 10 in June; the four additions are `request_logs`, `runtime_settings`, `llm_call_logs`, `ragas_evaluation_results`). Enums (`RiskLevel`, `InvestigationStatus`, `WorkState`) are DB-level Enum columns.

9.2 Indexing

Deliberate composite and single indexes: `idx_transaction_vendor`, risk, status, created_at on investigations; phase/round/source indexes on children; sequence and event indexes on the audit log; assigned/status on the queue. `request_logs` and `llm_call_logs` add `(investigation_id, created_at)` composite indexes. Hot query paths are covered.

9.3 Integrity, transactions, migrations

- Normalization ~3NF; JSON columns (citations, details, state_json, embedding) are intentional denormalization for agent-shaped payloads.
- Session-per-request; the executor commits per phase so checkpoints and partial progress survive a crash.
- Audit chain: `hash = sha256(prev_hash | canonical(payload))` with a sequence — a real tamper-evident chain (a test tampers a row and confirms detection).
- Migrations: **5 Alembic revisions**; production uses Alembic only (`create_all` no longer runs in prod — the June "split schema authority" issue is fixed).
- Locking: still neither optimistic nor pessimistic; concurrent re-runs and queue upserts can race under load (R12).

9.4 Bottlenecks

Single Postgres is the scaling ceiling; `state_json` checkpoints per phase can bloat (mitigated by the cleanup beat task); embeddings stored as JSON, not a pgvector vector column, so similarity search is Python cosine over the corpus rather than an ANN index — fine at the current small corpus, but it must migrate to `vector` + IVFFlat/HNSW before scaling retrieval. No soft-delete; `DELETE /intake/imported` is a hard delete.

Phase 10 — Backend Walkthrough

- **Controllers** — 14 APIRouters under `/api/v1`, each with prefix and tags; DI via `Depends` for `get_db_session`, `get_current_user`, and `require_elevated_role`.
- **Validation** — Pydantic v2 DTOs with field constraints.
- **AuthN** — OAuth2 password bearer to JWT. **AuthZ** — now partial: `require_elevated_role` gates destructive investigation routes; review approve/reject/escalate still use only `get_current_user` (R1 remains partly open).
- **Rate limiting** — an auth rate limiter now exists (`app/core/rate_limit.py`).
- **Services/domain** — `InvestigationExecutor`, the LLM gateway (`app/llm/service.py`), `EvidenceVerificationService`, the RAGAS judge, `eventstore`.
- **Background jobs** — Celery tasks + beat (cleanup daily; embedding sync).
- **Retries/timeouts** — Celery exponential backoff; LLM-layer tenacity retry on transient errors plus multi-provider fallback; Redis pings use 0.25s timeouts.
- **Idempotency** — `_reset_generated_outputs` makes re-execution idempotent; API POSTs are still not idempotency-keyed (R9).
- **Circuit breakers** — the LLM layer now retries and fails over; a formal breaker is still absent.
- **Exception handling** — a global handler with `request_id` correlation now exists; broad `except Exception` blocks remain in places (narrow them — S11).
- **Observability** — `/metrics` (Prometheus), optional LangSmith tracing, and `request_logs`/`llm_call_logs` telemetry. App logs are still `basicConfig` strings (`python-json-logger` present but not wired) — structured logging is a quick win.

Phase 11 — Frontend Walkthrough

- **Routing** — App Router; one route group with a shared `AppShell`; typed route table; an `unauthorized` segment now exists.
- **State** — server state in React Query (staleTime 60s); UI-only state in a small Context. No Redux/Zustand — correct.
- **Data/API** — `services/*` map snake_case DTOs to camelCase domain types through `api.ts` (anti-corruption layer).
- **Realtime** — `use-investigation-realtime` (WS to query invalidation, reconnect/backoff).
- **Forms** — react-hook-form + Zod.
- **Rendering/perf** — RSC by default; heavy widgets (ReactFlow graph) are code-split with skeletons; charts via Recharts.
- **A11y** — semantic nav with aria attributes; skeleton/empty/error states.
- **Gap** — still no `middleware.ts` route guard; an `unauthorized` page exists but server-side/route enforcement should be confirmed and completed (R16).

Phase 12 — Security Audit (updated status)

Posture: **fair** — improved from the June "weak," but not yet enterprise-grade for audit data. Status column reflects the current code.

#	Sev	Finding	Status now	Fix
S1	Critical to High	Authorization (RBAC): role existed but was unenforced	**Partially fixed** — `require_elevated_role` gates destructive investigation routes; review approve/reject/escalate still only require any authenticated user	Extend the elevated-role dependency to review and settings write actions
S2	Critical	Client-exposed credentials: `NEXT_PUBLIC_API_USERNAME/PASSWORD` ship in the browser bundle	**Open**	Remove; use a server-side login/BFF token exchange; never `NEXT_PUBLIC_` a secret
S3	High	Default admin seeded	**Partially mitigated** — prod requires a >=12-char `DEFAULT_ADMIN_PASSWORD`; seeding still defaults on	Disable seeding in prod; force first-run rotation
S4	High	`AUTH_REQUIRED=false` allows anonymous	**Mitigated in prod** — production forces `AUTH_REQUIRED=true`; still off in dev	Keep the prod guard; never allow anonymous on a deployed env
S5	High	No rate limiting / brute-force protection	**Partially fixed** — an auth rate limiter now exists	Extend limits to other sensitive routes; add lockout/backoff
S6	High	Prompt injection: untrusted text flows into prompts unsanitized	**Open**	Delimit/escape untrusted input, instruction-defense system prompts, validate output schema

#	Sev	Finding	Status now	Fix
S7	Medium	JWT HS256 shared secret; no refresh/rotation/revocation	**Open**	RS256 + JWKS, short-lived access + refresh + revocation; move to OIDC
S8	Medium	CORS/hosts wildcards permissive by default	**Mitigated in prod** — the prod validator rejects `*` for CORS and hosts	Keep explicit origins everywhere
S9	Medium	Secrets in plain `.env` / k8s Secret	**Open**	Vault/SSM/Sealed-Secrets; rotate keys; `.env` is git-ignored
S10	Low/Med	No field encryption, hard deletes, no GDPR tooling	**Open**	Encrypt sensitive columns; soft-delete + retention; data-subject export/erase
S11	Low	Broad `except Exception`; verbose error detail	**Partially fixed** — a global handler now hides internals (generic message + `request_id`) unless `DEBUG`; broad excepts remain internally	Narrow exceptions; keep correlation IDs
S12	Medium	WebSocket endpoint authentication	**Verify** — flagged historically as unauthenticated; confirm the current `/ws/investigations/{id}` validates a token on connect	Authenticate the WS handshake; reject unauthorized upgrades

Good practices present: bcrypt with SHA-256 pre-hash; parameterized ORM (SQL-injection safe); non-root Docker user; fail-fast prod config validation; tamper-evident audit chain; `WWW-Authenticate` on 401; rate-limited auth; global error handler with correlation IDs.

Compliance: the audit chain plus replay materially help SOC 2 / SOX evidence; GDPR (erasure, minimization, encryption) and comprehensive access-control auditing are not yet met.

Phase 13 — Production Readiness (re-scored)

Capability	Status	Notes
Health checks	Yes	`/health`, `/health/detailed`; Docker + k8s probes
Graceful shutdown	Yes	lifespan disposes engine; Celery time limits
Horizontal scaling	Yes (API + worker)	API replicas; workers scale out; Postgres single point
Async offload	Yes	Celery worker tier; in-process fallback
Migrations	Yes	Alembic; prod no longer runs `create_all` (fixed)
Containerization	Yes	Multi-stage, non-root, healthcheck
Config validation	Yes	Prod safety checks fail-fast
Retries/backoff	Yes	Celery backoff + LLM-layer tenacity retry + multi-provider fallback
Metrics	**Yes (was No)**	`/metrics` via prometheus-fastapi-instrumentator; request/LLM telemetry tables
Distributed tracing	Partial (was No)	Optional LangSmith on LLM/graph calls; no OTel across API and worker
Structured logging	Partial	Request/LLM audit rows in DB; app logs still plain strings
CI/CD	**Yes (was No)**	GitHub Actions: lint, typecheck, tests, build, image publish, deploy
IaC	Partial (was No)	k8s YAML + production Compose + Railway config; no Terraform/Pulumi
Rate limiting	**Yes (was No)**	Auth rate limiter
RBAC	Partial (was No)	Elevated-role gate on destructive investigation routes
Disaster recovery / backups	Partial (was No)	Documented runbook in `Docs/` (`BACKUP_RESTORE.md`, `DISASTER_RECOVERY.md`); automation still environment-specific

Capability	Status	Notes
Rollback / blue-green / canary	Partial	Image-SHA rollback + RollingUpdate; no canary/blue-green
Feature flags	Yes	Extensive env flags
Secrets management	No	Plain env / k8s Secret
Load testing / SLOs	No	None evident
Autoscaling (HPA)	Partial	Fixed replicas; no HPA manifest

Production readiness score: 69 / 100 (was 58). Sub-scores: Architecture 9/10 | Code quality 8/10 | Testing 6/10 | Security 5/10 | Observability 6/10 | Scalability 7/10 | Data/DR 5/10 | Deploy/CI-CD 7/10 | AI completeness 7/10 | Docs 9/10. Reading: the June foundation with much of the production-hardening (metrics, CI/CD, LLM resilience, rate limiting, partial RBAC, real evaluation, documentation) now in place; the remaining lift is comprehensive RBAC, browser-credential removal, prompt-injection defense, learned embeddings + ANN, distributed tracing, and backup/DR automation.

Phase 15 — Architecture Defense (enterprise answers)

- **Why FastAPI not Django?** The workload is async-first (WebSocket streaming, concurrent LLM calls) and contract-driven (Pydantic/OpenAPI). Django's ORM/admin batteries are weight we do not need, and its sync core fights the realtime path.
- **Why Postgres not Mongo?** It is an audit system: multi-row integrity, ACID, and a tamper-evident chain matter more than schema flexibility. Postgres also gives JSONB for agent-shaped payloads and pgvector for retrieval.
- **Why a modular monolith not microservices?** One team, correlated change cadence, shared domain model; the one component that needs independent scale (agent execution) is already a separate Celery worker tier; clean module seams make later extraction cheap.
- **Why Redis for everything?** Broker, result backend, pub/sub bus, and cache in one dependency at our scale. Redis Streams or Kafka is the upgrade path when audit-event throughput demands it.
- **Why LangGraph not CrewAI/AutoGen?** Our control flow is a graph with a loop — a debate cycle and a conditional verifier-driven re-run. LangGraph models nodes, shared typed state, and conditional edges explicitly, which is testable.
- **Why Claude, and how do you avoid lock-in?** Strong reasoning and reliable structured output, with a natural cheap/strong tier. And we are no longer single-vendor: the LLM gateway routes across five providers with a configurable fallback order, a response cache, and retry — vendor lock-in is low and outages degrade gracefully.
- **Why JWT not sessions?** Stateless fits the SPA + WebSocket + horizontal-API design. For enterprise I would graduate to OIDC with short-lived access + refresh + revocation.
- **Why REST+WS not GraphQL/gRPC?** CRUD is REST-shaped and the realtime need is server-push, which WebSocket serves directly.
- **How do you observe it in production?** Prometheus scrapes `/metrics`; per-request and per-LLM-call telemetry lands in Postgres; LangSmith can trace the agent graph. The next step is OpenTelemetry spans across API and worker for end-to-end traces.

Phase 16 — Improvement Roadmap (status-updated)

P0 = before any real deployment. "Status" reflects progress since June.

ID	Sev	Issue	Status	Fix	Effort
R1	P0	Comprehensive RBAC	**In progress**	Extend `require_elevated_role` to review/settings writes	S
R2	P0	Client-exposed creds	Open	Real login + server-side token exchange (BFF)	M
R3	P0	Real AI + retrieval	**Largely done**	Real agents wired (flag-gated); hybrid retrieval + multi-provider gateway + RAGAS judge implemented; remaining: learned embeddings, default-on rollout gated by evals	L
R4	P0	Observability	**Partially done**	`/metrics` + LangSmith + telemetry done; add OTel spans and JSON logs + correlation IDs	M
R5	P1	CI/CD	**Done**	GitHub Actions pipeline shipped; add image scanning (Trivy)	M

ID	Sev	Issue	Status	Fix	Effort
R6	P1	Prompt-injection defense	Open	Input delimiting, instruction defense, schema validation, treat retrieved text as data	M
R7	P1	Rate limiting	**Partially done**	Auth limiter shipped; extend to other routes	S
R8	P1	Schema authority split	**Done**	Prod is Alembic-only; `create_all` dev/test only	S
R9	P1	Idempotency on POST	Open	Idempotency-Key header + dedupe	S
R10	P2	No repository layer	Open	Extract repositories; thin controllers	M
R11	P2	Embeddings as JSON, not vector	Open	Migrate to pgvector `vector` + IVF-Flat/HNSW; learned embedding model	M
R12	P2	No concurrency control	Open	Optimistic `version_id` or row locks on re-run/queue	M

ID	Sev	Issue	Status	Fix	Effort
R13	P2	Multi-provider fallback / cost guard	**Largely done**	Fallback + cache + retry + cost telemetry shipped; add per-case token budgets and a circuit breaker	M
R14	P2	Broad `except` everywhere	**Partially done**	Global handler + typed errors added; narrow internal excepts	M
R15	P3	Soft-delete / retention / DR runbook	**Partially done**	DR/backup runbooks documented; add soft-delete + automated backups	M
R16	P3	Frontend route guard	Open	Next `middleware.ts` auth gate / server checks	S
R17	P3	Stub/real pipeline drift	Open	Contract tests asserting both paths emit identical event shapes	S

Sequence now: finish R1/R2/R4/R6 (authz + credentials + tracing + injection) then R9/R10/R11/R12 (structure + resilience) then R15/R16/R17 (lifecycle + polish). R3, R5, R8, R13 are substantially delivered.

Phase 14 — Interview Preparation (curated)

80 high-yield questions. Format: Q — ideal answer (mistake to avoid). Items whose answer changed since June are marked **[Updated]**.

Project Overview

1. **One sentence?** A real-time multi-agent AI platform that investigates financial-audit transactions by having LLM agents debate risk, grounding every verdict in evidence, and routing uncertain cases to humans with a tamper-evident audit trail. (Not a chatbot or a fraud classifier.)
2. **Why "professional skepticism"?** The auditing principle that conclusions must be evidenced and challenged; the architecture encodes it (debate + verifier + audit).
3. **[Updated] State of the project?** v0.1.0; full pipeline + UI wired; runs on deterministic stubs by default; the real-agent path, a multi-provider LLM gateway (fallback/cache/retry), hybrid retrieval, real-time RAGAS evaluation, metrics, and CI/CD are now implemented; pilot-grade at 69/100. Remaining: comprehensive RBAC, browser-credential removal, prompt-injection defense, learned embeddings + ANN, distributed tracing. (Do not overclaim "production AI"; do not under-claim by repeating the old "RAG not built.")
4. **Who pays and for what?** Analysts/reviewers/partners; scale, consistency, and defensibility (audit chain + replay).
5. **Walk a transaction.** intake, execute, evidence, debate, adjudication, verification, (retry?), confidence gate, report/human review, close — streamed live. (Do not skip the verifier loop — it is the differentiator.)

Architecture

6. **What pattern?** Modular monolith, layered + hexagonal touches, agentic workflow engine, event-driven realtime, event-sourcing-flavored audit — a deliberate hybrid. (Not microservices.)
7. **Monolith over microservices?** One team, correlated change, shared model; the scale-sensitive part (agents) is already a separate worker tier; seams make extraction cheap.
8. **Worker-to-API realtime?** It cannot reach in-memory WS connections, so it publishes to Redis pub/sub; the API subscribes and forwards. (Not shared memory.)
9. **Where is event-sourcing?** Append-only hash-chained `audit_log` + per-phase checkpoints enabling replay — not full ES/CQRS.
10. **[Updated] Scale to 10x?** Scale worker replicas; add Postgres read replicas / partition the audit log; move events to Redis Streams/Kafka; the LLM gateway already caches and fails over; HPA on queue depth; and you now have `/metrics` to drive that autoscaling.
11. **SPOFs?** Postgres, Redis, and the LLM providers — though provider failure now degrades via fallback rather than hard-failing.
12. **Why dual-path execution?** Graceful degradation — laptop with no broker, or k8s at scale; broker preflight avoids hangs. (Not dead code; drift mitigated by contract tests.)

Backend / Python

13. **Why FastAPI?** Async-first, contract-driven, typed docs. DI resolves `get_db_session`/`get_current_user`/`require_e` per request.
14. **DB session scope?** Session-per-request generator with `finally: close()`; background tasks open their own `SessionLocal`.
15. **Sync SQLAlchemy in async app?** A risk — blocking calls can stall the loop; mitigated because heavy work runs in Celery and `asyncio.to_thread` wraps agent calls; long-term move to async SQLAlchemy.
16. **LLM calls off the loop?** `await asyncio.to_thread(...)` in the executor; and the LLM gateway centralizes provider calls, cache, and retry.
17. **Pydantic v2 role?** Request/response validation + settings; DTOs decoupled from ORM.

18. **Config foot-gun prevention?** `@model_validator` rejects default secret/admin password/wildcard CORS in prod and requires a provider key for real agents.
19. **Explain `_extract_json`.** Defensive parse of LLM output — `json.loads`, then regex, then safe default — so a non-JSON response cannot crash adjudication.
20. **[Updated] Celery vs LLM retries?** Celery tasks retry with exponential backoff; the LLM gateway additionally retries transient provider errors with tenacity and then fails over to the next provider — two independent resilience layers.

Frontend / TypeScript

21. **React Query not Redux?** Server state needs caching/invalidation/dedupe; UI-only state lives in Context.
22. **Live updates without polling?** WS event to `queryClient.invalidateQueries(keys)` to refetch. Push-to-invalidate (refetch guarantees server truth).
23. **Anti-corruption layer?** `services/*` map snake_case DTOs to camelCase domain types.
24. **Bundle size?** RSC default + `dynamic(ssr:false)` code-splitting heavy widgets.
25. **Route group `(app)`?** Groups segments under one layout without affecting the URL.
26. **[Updated] Accessibility / auth?** Semantic nav, aria attributes, dedicated empty/error/loading states; an `unauthorized` route now exists, but a `middleware.ts` route guard is still the missing piece (R16).

Database

27. **Walk the model.** `investigations` aggregate root; cascade children; logical-only audit FK (must survive parent deletion); standalone users/embeddings plus telemetry tables.
28. **[Updated] Indexing?** Composite indexes on hot paths; `request_logs` and `llm_call_logs` add `(investigation_id, created_at)` composites. One to add: a partial index on `status='pending'` review items.
29. **Tamper-evident audit?** `hash = sha256(prev | canonical(payload))` + sequence; harden with external anchoring / WORM.
30. **Transaction boundaries?** Commit per phase so checkpoints and partial progress survive crashes.
31. **Concurrency control?** Still none — needs optimistic `version_id` or row locks for concurrent re-runs/queue upserts (R12).
32. **[Updated] pgvector usage?** Retrieval is implemented, but embeddings are deterministic feature-hash vectors stored as JSON and ranked by Python cosine + lexical overlap — not a pgvector `vector` column or ANN index yet. (Do not claim learned embeddings or ANN.)

AI / LLMs / Agents / RAG

33. **Multi-agent vs one prompt?** A single model hallucinates and anchors; adversarial debate
    - a separate grounding verifier produce defensible, evidenced verdicts.
34. **Agent graph + loop?** supervisor, evidence, challenger<->defender, adjudicator, verifier, END; conditional `route_debate`; outer re-run on ungrounded, bounded by `MAX_DEBATE_ROUNDS`/`MAX_VERIFICATION_RETRIES`.
35. **Design patterns?** Supervisor/router, adversarial debate, structured output, reflection/evaluator (verifier), human-in-the-loop gate. The IP is the verifier-driven bounded re-run.
36. **Prevent hallucination?** Three guards — debate, verifier rejection + re-query, no-fabrication external benchmarks — plus structured-output parsing.
37. **[Updated] Is there RAG?** Yes, a working hybrid retriever (chunking + feature-hash embeddings + cosine/lexical ranking over a policy KB, indexed at startup, `/knowledge/search`). The honest caveat: deterministic embeddings and JSON storage, not a learned model or ANN index. (The old "scaffolded only" answer is now wrong.)
38. **Model tiering (Sonnet/Haiku)?** Reasoning agents use the strong model; the verifier uses the cheaper model at low temperature. Cost/token telemetry now lands in `llm_call_logs`.

39. **Structured output + risks?** Prompt-for-JSON + defensive parse; better = JSON/tool mode + schema validation.
40. **Prompt-injection defense?** Still none — untrusted vendor/registry text enters prompts; fix with delimiting, instruction defense, treat-retrieved-as-data, output validation (S6).
41. **[Updated] Evaluate the agents?** A real-time RAGAS LLM judge now scores each investigation (faithfulness/grounding) and persists results; next add a labelled regression set gated in CI, plus online metrics (override rate, escalation rate, time-to-verdict).
42. **[Updated] Cost/latency controls?** Tiering + max-tokens + temperature, plus a response cache, multi-provider fallback, and per-call cost telemetry are implemented; still to add: per-case token budgets and a formal circuit breaker.
43. **[Updated] If the LLM is down?** The gateway retries transient errors and fails over to the next configured provider; Celery also retries; add a circuit breaker and degrade to the human queue on total failure.
44. **Determinism?** Cannot be fully deterministic; mitigations — low temp for the verifier, structured output, checkpoints/replay, and a stub path for tests.

Security

45. **[Updated] Biggest risk?** Authorization is now partial — destructive investigation routes require an elevated role, but review approve/reject/escalate do not yet; and the browser still ships dev credentials (S2). Finish RBAC and remove client creds.
46. **Frontend auth problem?** `NEXT_PUBLIC_*` credentials ship to the browser; no route guard. Fix with a BFF/login form + server token exchange + Next middleware.
47. **[Updated] JWT critique + WS auth?** HS256 shared secret, no refresh/revocation — upgrade to RS256/JWKS or OIDC. Confirm the WebSocket handshake authenticates the token on connect (S12) — historically it did not.
48. **SQL injection?** Low — parameterized ORM; the one raw spot is a constant `text("SELECT 1")` health check.
49. **Audit trustworthiness?** Hash chain + sequence; harden with external anchoring/WORM and signed events.
50. **Secrets management?** Plain `.env` / k8s Secret + GitHub Actions Secrets today; move to Vault/SSM/Sealed-Secrets + rotation.

Performance / Concurrency / Distributed Systems

51. **Where is the latency?** LLM calls dominate; debate rounds x retries multiply it; offloaded async so UX stays live.
52. **Race conditions?** Concurrent re-runs and queue upserts lack locking; idempotent reset helps but is not a lock (R12).
53. **WS scaling across replicas?** Each replica holds its own connections; Redis pub/sub fans events to all; each forwards to its clients; no sticky sessions.
54. **Backpressure?** Celery time limits; needs queue-depth autoscaling + DLQ.
55. **Idempotency?** `_reset_generated_outputs` makes execution idempotent; API creates are not idempotency-keyed (R9).
56. **Exactly-once vs at-least-once?** Celery is at-least-once; tasks must be idempotent; audit writes dedupe on (investigation, sequence).

Cloud / Docker / Kubernetes / CI-CD

57. **Walk the Dockerfile.** Multi-stage (system-wide deps so the non-root user can run console scripts), non-root user, healthcheck, `--workers 4`. Improve: pin base by digest, distroless, read-only FS.
58. **docker-compose services?** postgres, redis, eventstore, api, worker, beat, flower, and pgadmin (tools profile) — full local stack with healthchecks + `depends_on` conditions.
59. **[Updated] k8s highlights/gaps?** Namespace, ConfigMap/Secret, api Deployment replicas=3

RollingUpdate + probes + Prometheus annotations — and the `/metrics` endpoint the annotations scrape now exists. Gaps: no HPA, raw Secret. Make it production-grade with HPA on queue depth, NetworkPolicies, PodSecurity, sealed secrets, PDBs.

60. **[Updated] CI/CD?** It exists: GitHub Actions runs ruff + pytest, eslint + tsc + vitest + build, publishes images to GHCR, and deploys over SSH on dispatch. Next: add image scanning (Trivy) and canary via Argo/Helm.
61. **`--workers 4` in Docker but k8s replicas — double scaling?** Yes; prefer one worker per container in k8s and scale via replicas for clean autoscaling/observability.

Testing

62. **Strategy?** pytest on sqlite with stubbed agents (fast, deterministic); vitest + Testing Library on the front end; ruff correctness gate in CI.
63. **Under-tested?** Real-agent path, prompt-injection, concurrency, WS forwarding, e2e. Add contract tests asserting stub/real emit identical event shapes.
64. **Test non-deterministic AI?** Assert invariants (valid JSON, grounding flags, state transitions), not exact text; record/replay; eval harness with thresholds gating merges.

System Design / Trade-offs / Production

65. **Design the RAG upgrade.** Ingest policies/contracts, chunk (semantic, ~512-1024 tokens, overlap), embed with a learned model, pgvector HNSW, hybrid (BM25 + vector), rerank, inject top-k with citations into the Evidence agent — replacing today's deterministic feature-hash vectors.
66. **[Updated] Add observability — what remains?** `/metrics`, request/LLM telemetry, and optional LangSmith are in; add OpenTelemetry traces spanning API to Celery to LLM, plus JSON logs + correlation IDs and SLO alerts. The golden signal is grounding/escalation rate.
67. **[Updated] Cut LLM cost 50 percent?** The response cache and provider routing are in place; add semantic caching of evidence/debate, route easy cases to smaller models, cut debate rounds when early confidence is high, and use the cost telemetry (now populated) to target spend.
68. **Multi-tenancy?** Not present; add `tenant_id` scoping + row-level security + per-tenant rate/cost budgets.
69. **DR plan?** PG PITR backups, ESDB backups, Redis reconstructible; documented RTO/RPO + restore runbook (now in `Docs/`).
70. **What breaks first at 100x?** Postgres write contention + LLM rate limits — and you would now see it in the metrics you already expose.

Debugging / Failure Scenarios

71. **Stuck in `agent_debate`?** Check worker logs/Flower, Celery task state, LLM timeouts/retries, the `MAX_DEBATE_ROUNDS` termination, DB checkpoint rows.
72. **Live updates stopped?** WS open? `USE_REDIS_EVENTS` on? Redis reachable? forwarder running? client invalidation firing? In single-process mode the in-proc broadcast covers it.
73. **Verifier always rejects?** Prompt/model issue or genuinely missing corroboration; check `ungrounded_claims`, evidence artifacts, stub vs real path, threshold logic; bounded by `MAX_VERIFICATION_RETRIES` then escalate.
74. **Adjudicator returns non-JSON?** `_extract_json` falls back to a default medium-risk object; verdict still flows. Better: reject + retry with schema enforcement.
75. **Double-submitted case?** Two investigations created (no idempotency key) — add one (R9).

Leadership / Behavioral

76. **A trade-off you made?** Default stub agents + flags — chose demoability/testability/ offline-dev over "real AI on by default," accepting stub/real drift risk (mitigated by contract tests).
77. **Cut to ship in two weeks?** Keep security (finish RBAC) + one real-agent path on a narrow case type; defer RAG breadth and analytics polish.

78. **[Updated] Defend "AI off by default"?** It is a maturity gate, not a flaw — the pipeline is proven end-to-end with deterministic behavior, the real path and a multi-provider gateway with fallback/cache/retry are implemented, and a real-time evaluation judge is wired; flipping it on is config gated on eval thresholds.
79. **Biggest technical risk?** Hallucinated verdicts reaching sign-off — bounded by the verifier + human gate + audit; residual risk is prompt injection + not-yet-fully-evaluated real agents.
80. **Rebuild day one?** Observability + RBAC from the start (both now partly in), a repository layer, a real eval harness in CI, async DB, and a thin real-agent slice before breadth.

Phase 17 — Presentation Tracks

5-minute project explanation

"Skeptic Engine investigates financial-audit transactions with a crew of AI agents that argue about risk instead of a single model guessing. A transaction comes in; an Evidence agent gathers context and retrieves relevant policy; a Challenger and a Defender debate the worst-case and best-case readings; an Adjudicator renders a risk verdict with a confidence score; and — the key part — a Verifier checks that every claim is grounded in evidence. If it is not, a Supervisor sends the case back for more corroboration before any human sees it. Low-confidence or high-risk cases go to a human-review queue; everything is written to a tamper-evident, hash-chained audit log and streamed live to the UI. It is a Next.js front end and a FastAPI backend with a LangGraph agent core, a multi-provider LLM gateway with fallback and caching, Celery for async work, Postgres for data, and Redis for realtime, with Prometheus metrics and a CI/CD pipeline. The philosophy is trust over cleverness. Today it is a strong v0.1.0: the pipeline and UI work end-to-end, the real-LLM path, hybrid retrieval, and a real-time evaluation judge are implemented behind feature flags, and the remaining work is finishing RBAC, removing browser credentials, prompt-injection defense, and learned embeddings."

15-minute technical walkthrough (outline)

1. Problem and thesis (2m) — audit needs evidenced, challengeable, traceable verdicts.
2. Architecture (3m) — modular monolith, API + Celery worker, Postgres/Redis/EventStoreDB; the HTTP/WS seam.
3. Agent pipeline (4m) — LangGraph state machine, debate loop, verifier-driven re-run, confidence gate.
4. Realtime (2m) — worker to Redis to API to WebSocket to React-Query invalidation.
5. Resilience and evaluation (2m) — multi-provider LLM gateway (fallback/cache/retry), real-time RAGAS judge, `/metrics`.
6. Honest gaps (2m) — learned embeddings, comprehensive RBAC, prompt-injection defense.

30-minute deep dive (outline)

Add: a live trace of `execute_investigation` through each phase; the stub-vs-real dual path; the ADR table (Phase 7) with defenses; the security findings (Phase 12) and the RBAC/credential fixes; the re-scored readiness scorecard (Phase 13); the retrieval upgrade (Phase 16, R11); and the roadmap sequencing.

Deliverables Index

Requested deliverable	Section
Executive Summary	Front matter
Architecture Document	Phases 3-5
Code Walkthrough Guide	Phases 4-5
Technology Decision Record (ADR)	Phases 7 and 15
API Documentation	Phase 10 + route map (Phase 1)
Database Documentation	Phase 9
AI Architecture Documentation	Phase 8
Production Readiness Report	Phase 13
Security Audit Report	Phase 12
Performance Review	Phases 10/13/16
Improvement Roadmap	Phase 16
Senior / Principal Interview Guide	Phase 14
Presentation tracks	Phase 17
Revision / change log	Front matter (Revision Notes)

Prepared by re-verifying the source at HEAD `cced496` (original at `2c9cfca`). Every claim is grounded in files cited inline; where a capability is absent or partial it is stated as such rather than assumed. Figures not measured in the codebase (token cost, latency) are called out as unmeasured.